

Recurrent Neural Networks: RNN, LSTM and GRU

K.Kadri

DVRC

January 12, 2026

Why Recurrent Neural Networks?

Key Motivation

Many real-world data are **sequential**:

- Time series
- Natural language
- Signals and trajectories

Limitation of Feedforward Networks

Standard neural networks assume:

- Fixed-size input
- No temporal dependency

Core Idea

RNNs introduce a **hidden state** acting as a memory of past inputs.

Vanilla Recurrent Neural Network

Hidden State Update

At time step t :

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h)$$

- $\mathbf{x}_t$: input at time t
- $\mathbf{h}_t$: hidden state (memory)
- ϕ : activation function (tanh, ReLU)

Key Limitation

Gradients may vanish or explode during backpropagation through time (BPTT).

Vanishing Gradient Problem

Backpropagation Through Time (BPTT)

Gradients propagate through repeated multiplications:

$$\prod_{k=1}^t \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-1}}$$

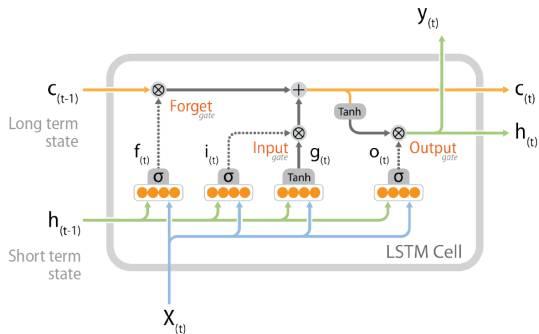
Problem

- Gradients $\rightarrow 0$ (vanishing)
- Long-term dependencies cannot be learned

Solution Direction

Introduce **gating mechanisms** to control information flow.

Long Short-Term Memory (LSTM)



Long short-term memory (LSTM)¹

Gated recurrent unit (GRU)²

$$\begin{aligned}
 f_{(t)} &= \sigma(W_{xf}^T X_{(t)} + W_{hf}^T h_{(t-1)} + b_f) \\
 i_{(t)} &= \sigma(W_{xi}^T X_{(t)} + W_{hi}^T h_{(t-1)} + b_i) \\
 g_{(t)} &= \tanh(W_{xg}^T X_{(t)} + W_{hg}^T h_{(t-1)} + b_g) \\
 o_{(t)} &= \sigma(W_{xo}^T X_{(t)} + W_{ho}^T h_{(t-1)} + b_o) \\
 c_{(t)} &= f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)} \\
 y_{(t)} &= h_{(t)} = o_{(t)} \otimes \tanh(c_{(t)})
 \end{aligned}$$

with :

$X_{(t)} \in \mathbb{R}^d$	input vector
$f_{(t)} \in \mathbb{R}^h$	forget gate's activation vector
$i_{(t)} \in \mathbb{R}^h$	input gate's activation vector
$o_{(t)} \in \mathbb{R}^h$	output gate's activation vector
$g_{(t)} \in \mathbb{R}^h$	current entry vector
$h_{(t)}, y_{(t)} \in \mathbb{R}^h$	hidden state or output vector
$c_{(t)} \in \mathbb{R}^h$	cell state vector
$\otimes$	Hadamard product
σ	sigmoid function
W_k	weights matrix
b_k	bias vector

Figure: Detailed mechanism of an lstm cell. (adapted from : Sepp Hochreiter, Jürgen Schmidhuber, (1997) [LSTM])

Long Short-Term Memory (LSTM)

Main Innovation

LSTM introduces a **cell state** $\mathbf{c}_t$ that allows:

- Long-term memory
- Controlled information flow

Key Mechanism

Three gates regulate information:

- Forget gate
- Input gate
- Output gate

Forget Gate

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Input Gate

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

Candidate Memory

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

LSTM Cell State Update

Cell State Evolution

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

- $\odot$: element-wise multiplication
- Forget gate controls memory removal
- Input gate controls memory injection

Key Advantage

Gradients can flow through $\mathbf{c}_t$ almost linearly.

LSTM Output Computation

Output Gate

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

Hidden State

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Interpretation

The hidden state is a filtered view of the internal memory.

Gated Recurrent Unit (GRU)

Design Philosophy

GRU simplifies LSTM by:

- Removing the explicit cell state
- Merging gates

Key Advantage

Fewer parameters, faster training, comparable performance.

Update Gate

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

Reset Gate

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

GRU Hidden State Update

Candidate Activation

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t])$$

Final State

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Interpretation

The update gate interpolates between past and new information.

Model Comparison

Summary

Model	Memory	Gates	Complexity
RNN	Weak	None	Low
LSTM	Strong	3	High
GRU	Strong	2	Medium

Rule of Thumb

- Short sequences → RNN
- Long dependencies → LSTM
- Limited resources → GRU

Key Insight

Recurrent models are not about time, but about **information persistence**.

Mental Model

- RNN: memory as a fragile echo
- LSTM: memory as a protected conveyor belt
- GRU: memory as a controlled interpolation